Team ID: SWTID-2026-5355
Team size : 4
Team Head : Sharmila Dayana.S
Team member : Nandhini.M
Team member: pavithra.P
Team member: Nishiya.M

Project Overview Document: Contact Manager API Backend

1. Project Overview

The Contact Manager API is a secure, RESTful backend service designed to manage user authentication and personal contact directories. It allows users to register, log in securely, and perform full CRUD (Create, Read, Update, Delete) operations on their specific contacts, ensuring data privacy and integrity through token-based authorization.

2. Key Objectives

Implement Secure Authentication: Utilize JSON Web Tokens (JWT) to securely authenticate users and protect private API routes.
Build RESTful Routing: Create clean, standard HTTP endpoints for resource management.
Database Integration: Connect and interact with a NoSQL database (MongoDB) to persist user and contact data.
Error Handling & Validation: Ensure robust server responses for bad requests, missing tokens, and invalid data.

3. Target Users

Primary: End-users (e.g., resort staff or customers) who need a secure system to store, search, and manage their address books.
Secondary: Frontend developers who will consume these APIs to build web or mobile user interfaces.

4. Pre-requisites & Knowledge Requirements

To develop and maintain this project, the following foundational knowledge is required:
Programming: JavaScript (ES6+ syntax).
Backend Environment: Node.js runtime environment.
Web Framework: Express.js for routing and middleware.
Database: MongoDB concepts and Mongoose ODM (Object Data Modeling).
Tools: VS Code, Command Line/PowerShell navigation, and API testing tools (Thunder Client or Postman).

5. Project Architecture & Tech Stack

The project follows the standard MVC (Model-View-Controller) architectural pattern (excluding the View, as this is purely an API).

Runtime: Node.js

Framework: Express.js

Database: MongoDB (using Mongoose for schema definitions)

Security: bcryptjs for password hashing, jsonwebtoken for API route protection.

Development Tools: nodemon for automatic server restarts, dote nv for environment variable management.

6. Database Architecture (MongoDB)

The database utilizes two primary collections linked by relationships:

User Schema:
_id: Unique identifier automatically generated.
name: String (required).
email: String (required, unique).

password: String (Hashed for security).
timestamps: Tracks creation and update times.

Contact Schema:

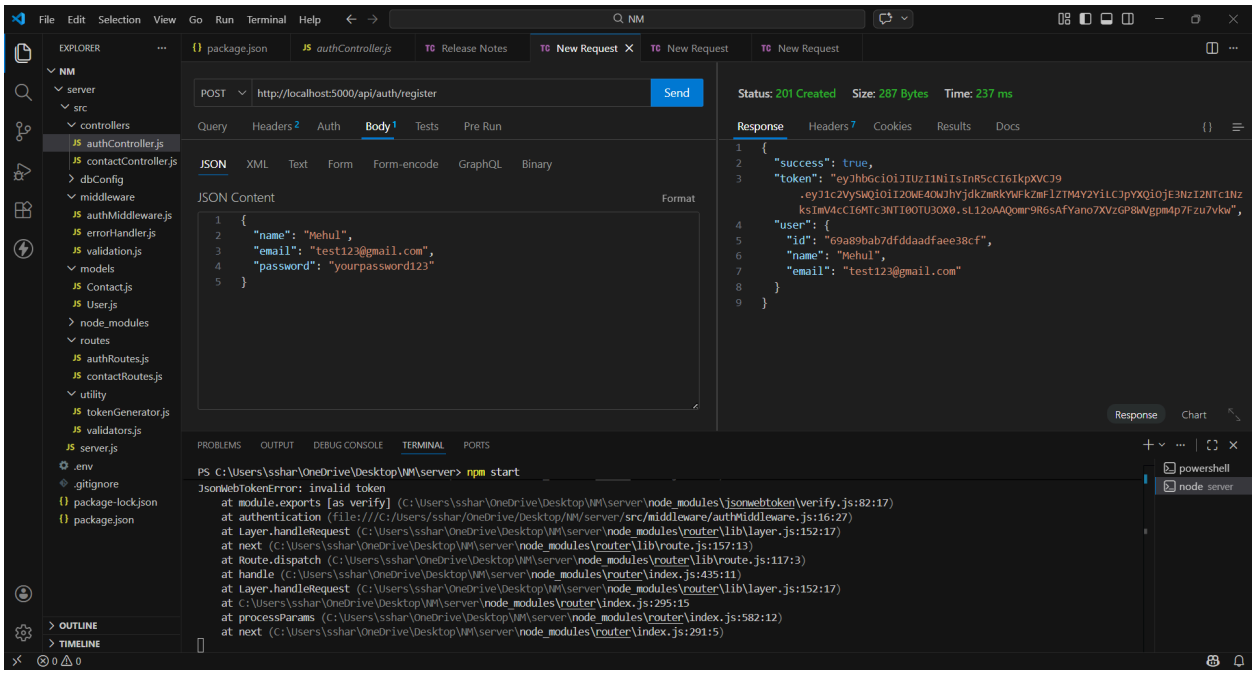
_id: Unique identifier.
userId: ObjectId (References the User who created the contact to ensure privacy).
name: String.
email: String.
phone: String.
tags: Array of strings (e.g., ["test", "work"]).

7. Features & API Endpoints

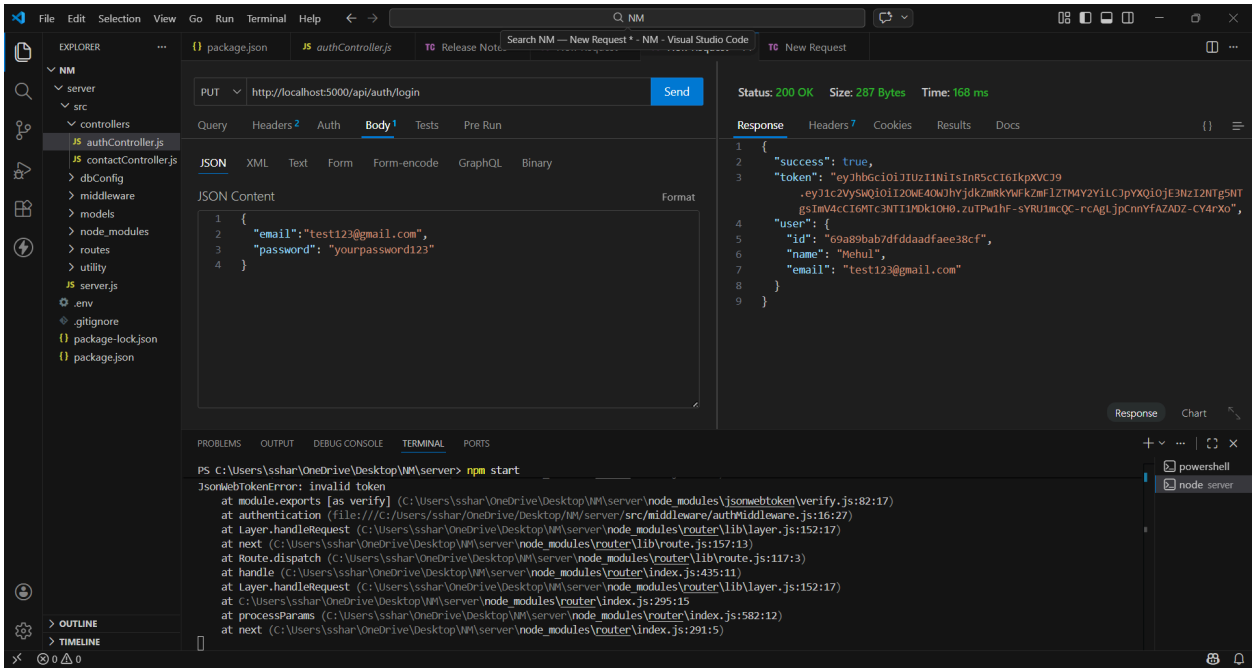
The system is divided into two main feature sets: Auth and Contacts.
Authentication Features (Public Routes):
Register User: POST /api/auth/register (Creates a new user and hashes their password).
Login User: POST /api/auth/login (Verifies credentials and returns a JWT).
Get Current User: GET /api/auth/me (Protected - Returns details of the currently logged-in user).

Contact Management Features (Protected Routes - Require JWT):

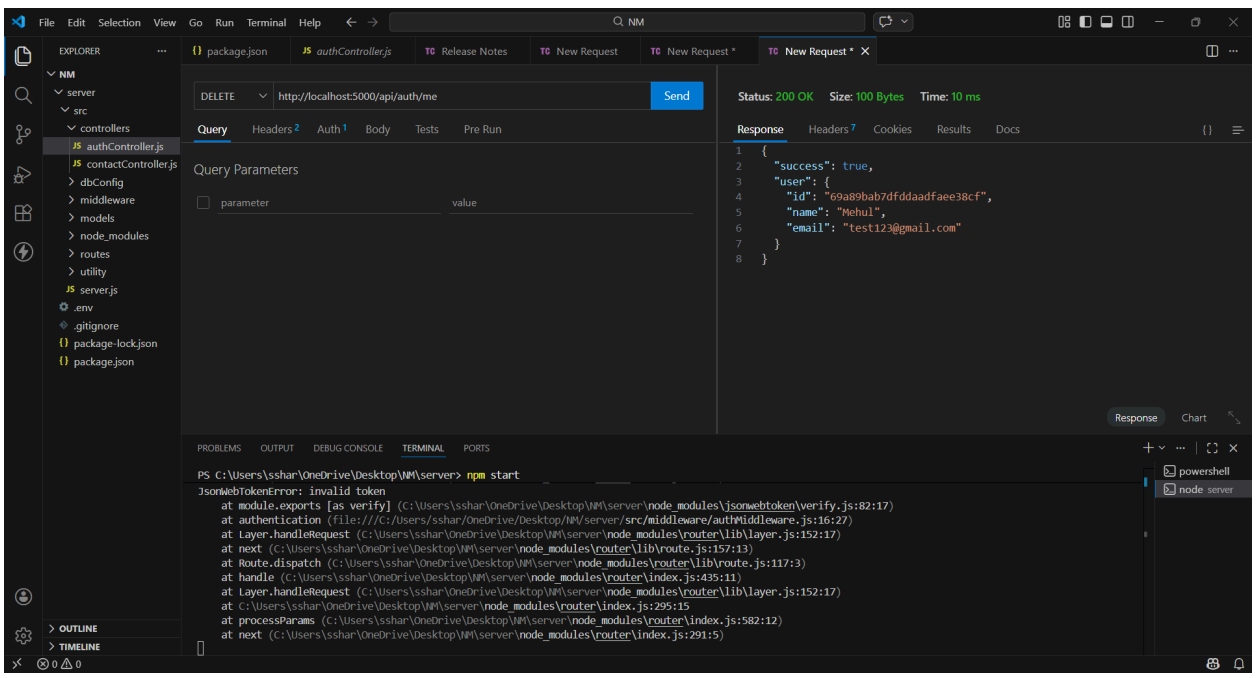
1. Create Contact: POST /api/contacts/



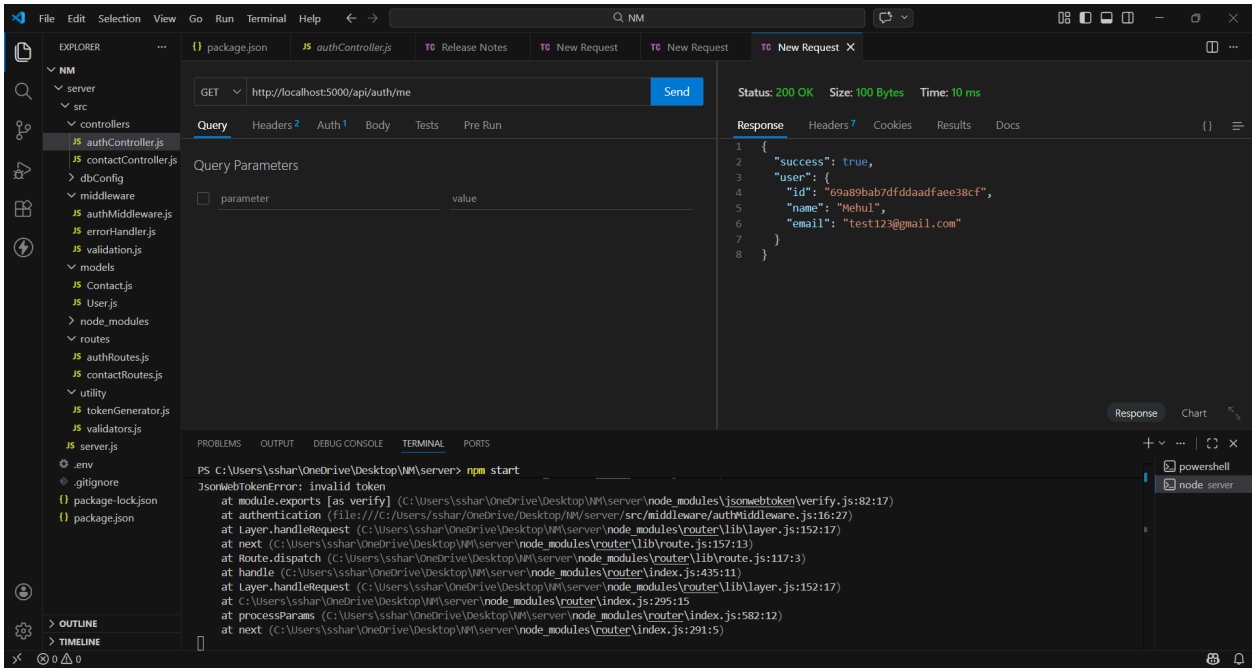
2.Update Contact: PUT /api/contacts/:id



3.Delete Contact: DELETE /api/contacts/:id



4.Search Contacts: GET /api/contacts/search?q={query}



8. User Flowchart

1.Start: User accesses the application.

2.Auth Node: Does the user have an account?

No: User navigates to Registration -> Enters details -> Server hashes password -> Saves to DB -> Redirects to Login.

Yes: User navigates to Login -> Enters credentials -> Server validates -> Generates JWT -> Returns Token to Client.

3.Action Node: User attempts to manage contacts.

Client attaches JWT as a Bearer Token in the request header.

Server's Auth Middleware validates the token.

Invalid Token: Returns 401 Unauthorized.

Valid Token: Grants access to Create, Read, Update, Delete, or Search contacts.

9. Project Setup and Configuration

Detailed steps required to spin up the local development environment:

1.System Preparation: * Install Node.js and MongoDB.
Ensure Windows PowerShell execution policies are set up correctly (Set-ExecutionPolicy RemoteSigned via Administrator) to allow script execution.

2.Project Initialization:

Extract project files and open the server folder in VS Code.

3.Install Dependencies:

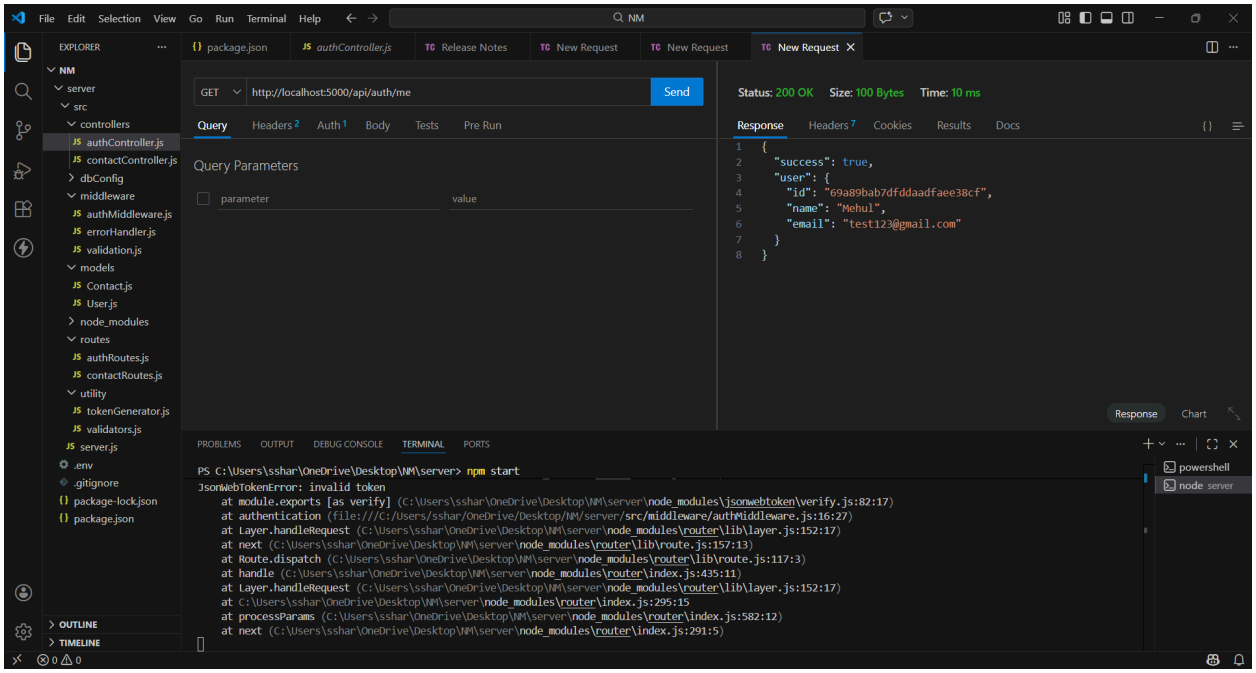
Run npm install in the terminal to download required packages from package.json.

4.Environment Variables:

Configure the .env file with the MongoDB connection string and a secret JWT key.

5.Run the Server:

Execute npm start (or npm run dev) to start the server using Nodemon.
Verify the terminal outputs "Server running on port 5000" and "MongoDB Connected".

6.API Testing:

Use the VS Code Thunder Client extension to simulate client requests and verify endpoint logic.